WEEK 6 · PHASE 3

PySpark/SQL ETL

Bronze → Silver Data Transformation

✥ Null handling & fillna strategies
✥ Type casting: dates, timestamps, numerics
✥ Column operations: add, rename, drop, filter
✥ Array & struct transformations
✥ MERGE INTO upsert patterns
✥ ANSI mode & error handling (Updated)

Table of Contents

1. Bronze → Silver Pipeline Architecture 3
2. Null Handling 4
3. Type Casting & ANSI Mode (Updated) 6
4. Column Operations 8
5. Array & Struct Transformations 10
6. MERGE INTO — Upsert Pattern 12
7. Data Cleaning Patterns 14
8. Practice Questions 15
9. Answer Key & Explanations 18
BONUS A: Advanced ETL Patterns 20
BONUS B: Deep Dive — Spark Internals for ETL 22
BONUS C: Real-World Lab Scenario 24
BONUS D: Mock Exam (8 Questions) 26
Cheat Sheet & Key Facts 28

1. Bronze → Silver Pipeline Architecture

The Medallion Architecture organizes data into three layers: Bronze (raw ingestion), Silver (cleaned/validated), and Gold (business-level aggregates). Week 6 focuses on the critical Bronze → Silver transformation step, where raw data is cleansed, typed, and deduplicated for downstream analytics.

Layer	Contents	Format	Transformation
Bronze	Raw, unmodified source	Delta (schema as-is)	None — exact copy
Silver	Cleaned, typed, deduplicated	Delta (enforced)	Null handling, casting, dedup
Gold	Business aggregates, KPIs	Delta (optimized)	Joins, aggregations, business logic

Typical Bronze → Silver Pipeline Skeleton

■ **Complete Bronze → Silver Pattern**

```
# 1. Read from Bronze
df_bronze = spark.table("catalog.bronze.orders")

# 2. Drop nulls in required fields
df = df_bronze.dropna(subset=["order_id","customer_id"])

# 3. Cast types
df = df.withColumn("order_date", to_date(col("order_date_str"), "yyyy-MM-dd")) \
       .withColumn("amount", col("amount_str").cast("double"))

# 4. Clean & standardize
df = df.withColumn("status", upper(trim(col("status"))))

# 5. Write to Silver (overwrite with schema evolution)
df.write.format("delta").mode("overwrite") \
   .option("overwriteSchema","true") \
   .saveAsTable("catalog.silver.orders")
```

2. Null Handling

Null values are the most common data quality issue in Bronze tables. Databricks provides multiple approaches depending on whether nulls should be dropped, replaced, or flagged.

2.1 dropna() — Remove Rows with Nulls

■ **dropna() Variants**

```
from pyspark.sql.functions import col

# Drop rows where ANY column is null (default)
df.dropna()

# Drop rows where ALL columns are null
df.dropna(how="all")

# Drop rows where SPECIFIC columns are null
df.dropna(subset=["order_id", "customer_id", "amount"])

# Drop rows with fewer than N non-null values
df.dropna(thresh=3)  # keep rows with ≥3 non-null cols
```

2.2 fillna() — Replace Nulls with Defaults

■ **fillna() Examples**

```
# Fill all string nulls with "unknown"
df.fillna("unknown")

# Fill specific columns with different values
df.fillna({"status": "PENDING", "amount": 0.0, "region": "N/A"})

# SQL equivalent: COALESCE
-- SELECT COALESCE(status, "PENDING"), COALESCE(amount, 0)
```

2.3 when() / otherwise() — Conditional Null Handling

■ **when/otherwise Pattern**

```
from pyspark.sql.functions import when, col, lit

# Replace null with computed value
df.withColumn("category",
    when(col("category").isNull(), lit("UNCATEGORIZED"))
    .when(col("category") == "", lit("EMPTY"))
    .otherwise(col("category"))
)

# SQL CASE equivalent
-- CASE WHEN category IS NULL THEN "UNCATEGORIZED"
--      WHEN category = "" THEN "EMPTY"
--      ELSE category END AS category
```

2.4 coalesce() — First Non-Null Across Columns

■ coalesce() Example

```
from pyspark.sql.functions import coalesce

# Return first non-null value across columns
df.withColumn("email",
    coalesce(col("email_primary"), col("email_secondary"),
             lit("no-email@example.com"))
)

# SQL: SELECT COALESCE(email_primary, email_secondary, "no-email")
```

■ EXAM
dropna() removes rows; **fillna()** replaces nulls in-place; **coalesce()** picks first non-null across columns; **when/otherwise** handles conditional logic.

3. Type Casting & ANSI Mode (Updated)

Bronze tables often store all columns as STRING (from CSV/JSON ingestion). Silver transformation casts them to proper types. However, parsing errors are handled differently depending on the **spark.sql.ansi.enabled** configuration.

3.1 ANSI Mode & Error Handling (NEW)

Setting	to_date() Behavior	to_timestamp() Behavior	Default in DBR 17+
spark.sql.ansi.enabled=true	Throws CAST_INVALID_INPUT error	Throws CANNOT_PARSE_TIMESTAMP error	✓ Default
spark.sql.ansi.enabled=false	Returns NULL silently	Returns NULL silently	Legacy

■■ WARN
Critical Update: In Databricks Runtime 17.0+ (Spark 4.0+), **spark.sql.ansi.enabled=true is the default**. This means to_date() and to_timestamp() **throw exceptions** on invalid input, not return NULL. To handle errors gracefully, use **try_to_date()** or **try_to_timestamp()**, or explicitly set spark.sql.ansi.enabled=false.

3.2 cast() — Explicit Type Conversion

■ cast() Syntax

```
from pyspark.sql.functions import col

# Cast string to numeric types
df.withColumn("amount", col("amount_str").cast("double"))
  .withColumn("quantity", col("qty_str").cast("integer"))
  .withColumn("user_id", col("user_id_str").cast("long"))
  .withColumn("is_active", col("active_str").cast("boolean"))

# Using explicit type classes
from pyspark.sql.types import DoubleType
df.withColumn("price", col("price_str").cast(DoubleType()))
```

3.3 Date & Timestamp Parsing (with Error Handling)

■ Safe Date Parsing

```
from pyspark.sql.functions import to_date, to_timestamp, col

# Using to_date() — may throw error if ANSI enabled
df.withColumn("order_date", to_date(col("date_str"), "yyyy-MM-dd"))

# Safer: use try_to_date() to handle parse errors gracefully
from pyspark.sql.functions import try_to_date
df.withColumn("order_date", try_to_date(col("date_str"), "yyyy-MM-dd"))

# Parse timestamp
df.withColumn("created_at",
    to_timestamp(col("ts_str"), "yyyy-MM-dd HH:mm:ss"))
```

Source Format	Target Type	Function	Format String
2024-01-15	DateType	to_date()	yyyy-MM-dd
01/15/2024	DateType	to_date()	MM/dd/yyyy
2024-01-15 14:30:00	TimestampType	to_timestamp()	yyyy-MM-dd HH:mm:ss
1705324200	TimestampType	cast("timestamp")	Unix epoch (no format)
"123.45"	DoubleType	cast("double")	N/A

■ TIP
Always validate after parsing with df.filter(col("order_date").isNull()).count(). If using ANSI mode, prefer try_to_date() and try_to_timestamp() to avoid unexpected exceptions.

4. Column Operations

4.1 Add / Derive New Columns

■ **Add & Derive Columns**

```
from pyspark.sql.functions import col, lit, current_timestamp, upper, trim, concat_ws

# Add literal column
df.withColumn("source_system", lit("SAP"))

# Derive from existing columns
df.withColumn("total_price", col("unit_price") * col("quantity"))
  .withColumn("status_upper", upper(trim(col("status"))))
  .withColumn("full_name", concat_ws(" ", col("first_name"), col("last_name")))
  .withColumn("_ingest_ts", current_timestamp())
```

4.2 Rename Columns

■ **Column Rename Patterns**

```
# Single column rename
df.withColumnRenamed("old_name", "new_name")

# Multiple renames via select + aliases
df.select(
    col("cust_id").alias("customer_id"),
    col("ord_dt").alias("order_date"),
    col("amt").alias("amount")
)

# Rename all to snake_case
import re
def to_snake(n): return re.sub(r"(?<!^)(?=[A-Z])", "_", n).lower()
df.toDF(*[to_snake(c) for c in df.columns])
```

4.3 Drop Columns

■ **Drop Patterns**

```
# Drop single column
df.drop("temp_column")

# Drop multiple columns
df.drop("col1", "col2", "col3")

# Drop by list (useful for dropping many)
cols_to_drop = ["_rescued_data", "_ingest_raw", "_source_path"]
df.drop(*cols_to_drop)
```

4.4 filter() / where() — Row Selection

■ **Filter & Where**

```
# PySpark filter with expressions
df.filter(col("status") == "ACTIVE")
df.filter((col("amount") > 100) & (col("region").isin("US","EU")))
df.filter(col("email").isNotNull())
```

```
# where() is an alias for filter()
df.where("status = 'ACTIVE' AND amount > 100")

# SQL equivalent
-- WHERE status = "ACTIVE" AND amount > 100
```

5. Array & Struct Transformations

5.1 explode() and explode_outer()

■ **Array Exploding**

```
from pyspark.sql.functions import explode, posexplode, explode_outer

# explode: one row per array element (drops null/empty arrays)
df.select("order_id", explode(col("items")).alias("item"))
  .select("order_id", "item.sku", "item.qty")

# posexplode: includes array index position
df.select("order_id", posexplode(col("items")).alias("pos","item"))
  .select("order_id", "pos", "item.sku")

# explode_outer: keeps row even if array is null/empty
df.select("order_id", explode_outer(col("items")).alias("item"))
```

Function	Null/Empty Array	Output	Use Case
explode()	Row dropped	1 row per element	Standard flattening
explode_outer()	Row kept (item=null)	1 row per element + null	Keep all orders
posexplode()	Row dropped	(index, element) per row	Position matters

5.2 struct() — Create Nested Column

■ **Struct Creation & Access**

```
from pyspark.sql.functions import struct

# Create struct from existing columns
df.withColumn("address",
    struct(col("street"), col("city"), col("zip")).alias("address")
)

# Access struct fields
df.select("address.city", "address.zip")
```

6. MERGE INTO — Upsert Pattern

MERGE INTO is the standard SQL command to upsert data into Delta tables. It combines INSERT, UPDATE, and DELETE operations in one atomic transaction.

■ Full MERGE INTO Syntax

```
MERGE INTO catalog.silver.customers AS target
USING catalog.bronze.customers_raw AS source
ON target.customer_id = source.customer_id

WHEN MATCHED AND source.updated_at > target.updated_at THEN
    UPDATE SET
        target.name = source.name,
        target.email = source.email,
        target.updated_at = source.updated_at

WHEN NOT MATCHED THEN
    INSERT (customer_id, name, email, created_at, updated_at)
    VALUES (source.customer_id, source.name, source.email,
            current_timestamp(), source.updated_at)

WHEN NOT MATCHED BY SOURCE AND target.status = "ACTIVE" THEN
    UPDATE SET target.status = "INACTIVE";
```

MERGE Clause	Condition	Allowed Actions
WHEN MATCHED	Key exists in source & target	UPDATE or DELETE
WHEN NOT MATCHED	Key only in source (new row)	INSERT
WHEN NOT MATCHED BY SOURCE	Key only in target (soft delete)	UPDATE or DELETE (Databricks)

6.1 PySpark merge() API

■ DeltaTable Python API

```
from delta.tables import DeltaTable

target = DeltaTable.forName(spark, "catalog.silver.customers")
target.alias("tgt").merge(
    source=df_bronze.alias("src"),
    condition="tgt.customer_id = src.customer_id"
).whenMatchedUpdateAll() \
 .whenNotMatchedInsertAll() \
 .execute()
```

■ EXAM
WHEN NOT MATCHED BY SOURCE is a Databricks-specific extension (not standard SQL). It handles rows that exist in target but NOT in source — useful for soft deletes.

7. Data Cleaning Patterns

■ String & Value Cleaning

```
from pyspark.sql.functions import trim, upper, lower, regexp_replace, col, round as spark_round

# String standardization
df.withColumn("status", upper(trim(col("status"))))
  .withColumn("country", lower(trim(col("country"))))

# Remove special characters (keep digits only)
  .withColumn("phone", regexp_replace(col("phone"), r"[^0-9]", ""))

# Standardize date format
  .withColumn("order_date", to_date(col("date_str"), "M/d/yyyy"))

# Round numeric
  .withColumn("amount", spark_round(col("amount"), 2))
```

Quick Reference: Common String Functions

Function	Purpose	Example
upper(col)	Convert to UPPERCASE	upper(col("status"))
lower(col)	Convert to lowercase	lower(col("country"))
trim(col)	Remove leading/trailing spaces	trim(col("name"))
length(col)	String length in characters	length(col("phone")) == 10
substring(col,pos,len)	Extract substring	substring(col("zip"),1,5)
split(col,pattern)	Split string to array	split(col("tags"),",")
concat_ws(sep,cols)	Concatenate with separator	concat_ws("-",col("y"),col("m"),col("d"))
regexp_replace(col,pat,rep)	Regex replace	regexp_replace(col("phone"),"[^0-9]","")

8. Practice Questions

1. A Bronze table has a "price" column stored as STRING. Which PySpark expression correctly converts it to DOUBLE?

A) df.withColumn("price", double(col("price")))
B) df.withColumn("price", col("price").cast("double"))
C) df.withColumn("price", to_double(col("price")))
D) df.cast(col("price"), DoubleType())

2. Which function drops rows only where ALL specified columns are null?

A) df.dropna(subset=["col1","col2"])
B) df.dropna(how="all", subset=["col1","col2"])
C) df.filter(col("col1").isNull() & col("col2").isNull())
D) df.dropna(thresh=0)

3. An engineer wants to keep all orders in the result even if their "items" array is empty or null after explode. Which function should they use?

A) explode()
B) posexplode()
C) explode_outer()
D) flatten()

4. MERGE INTO has a clause where a row exists in the target but NOT in the source. Which clause handles this?

A) WHEN NOT MATCHED
B) WHEN MATCHED AND source IS NULL
C) WHEN NOT MATCHED BY SOURCE
D) WHEN UNMATCHED

5. A column "email" may be null. Which expression returns the first non-null value from email_primary, email_secondary, then "unknown"?

A) when(col("email_primary").isNull(), col("email_secondary")).otherwise("unknown")
B) coalesce(col("email_primary"), col("email_secondary"), lit("unknown"))
C) fillna({"email_primary": col("email_secondary")})
D) nvl(col("email_primary"), col("email_secondary"))

6. (UPDATED) You call to_date("INVALID", "yyyy-MM-dd") in Databricks Runtime 17.0+ with default settings. What happens?

A) Returns NULL silently
B) Throws CAST_INVALID_INPUT exception
C) Returns the string "INVALID"
D) Returns epoch date 1970-01-01

Note: ANSI mode (spark.sql.ansi.enabled=true) is default in DBR 17+. Use try_to_date() for safe parsing.

7. Which PySpark expression renames column "cust_id" to "customer_id"?

A) df.rename("cust_id", "customer_id")
B) df.withColumnRenamed("cust_id", "customer_id")
C) df.select(col("cust_id").name("customer_id"))
D) df.withColumn("customer_id", col("cust_id")).drop("cust_id")

8. In the Medallion Architecture, which layer stores raw, unmodified source data exactly as received?

A) Gold

B) Silver
C) Bronze
D) Landing

9. Answer Key & Explanations

Q1: Answer B
col("price").cast("double") is the correct syntax. cast() is called on the Column object.

Q2: Answer B
dropna(how="all", subset=[...]) drops rows where ALL specified columns are null. Default how="any" drops if ANY specified column is null.

Q3: Answer C
explode_outer() keeps rows even when the array is null or empty (with null in the exploded column). Regular explode() drops such rows.

Q4: Answer C
WHEN NOT MATCHED BY SOURCE handles rows in the target table that have no matching key in the source. Databricks-specific extension for soft deletes.

Q5: Answer B
coalesce() returns the first non-null value from a list of columns/expressions.

Q6: Answer B
(UPDATED) In Databricks Runtime 17.0+, spark.sql.ansi.enabled=true (default). to_date() throws CAST_INVALID_INPUT on invalid input. Use try_to_date() for safe parsing.

Q7: Answer B
withColumnRenamed("old", "new") is the standard PySpark method. Option D also works but is verbose.

Q8: Answer C
Bronze is the raw ingestion layer — data is stored exactly as received (no transformations). Silver applies cleaning, typing, and deduplication.

BONUS **BONUS A: Advanced ETL Patterns** Production-Ready Code

Advanced ETL Patterns & Optimization

A.1 Deduplication with ROW_NUMBER()

■ **Window Function Deduplication**

```
from pyspark.sql.functions import row_number
from pyspark.sql.window import Window

# Keep first row per order_id (sorted by updated_at DESC)
window = Window.partitionBy("order_id").orderBy(col("updated_at").desc())
df.withColumn("rn", row_number().over(window)) \
   .filter(col("rn") == 1) \
   .drop("rn")
```

A.2 Complete Production Silver ETL

■ **Production Silver ETL**

```
from pyspark.sql.functions import *

df = spark.table("catalog.bronze.orders")

# 1. Remove fully null rows
df = df.dropna(subset=["order_id","customer_id","amount_str"])

# 2. Cast types (using try_to_date for ANSI-safe parsing)
from pyspark.sql.functions import try_to_date
df = (df
    .withColumn("amount", col("amount_str").cast("double"))
    .withColumn("quantity", col("qty_str").cast("integer"))
    .withColumn("order_date", try_to_date(col("date_str"), "yyyy-MM-dd"))
    .withColumn("created_at", try_to_timestamp(col("ts_str"), "yyyy-MM-dd HH:mm:ss"))
)

# 3. Clean strings
df = (df
    .withColumn("status", upper(trim(col("status"))))
    .withColumn("country", upper(trim(col("country_code"))))
    .withColumn("phone", regexp_replace(col("phone"), r"[^0-9+]", ""))
)

# 4. Handle nulls in optional fields
df = df.fillna({"region": "UNKNOWN", "discount": 0.0})

# 5. Derive new columns
df = (df
    .withColumn("total_price", round(col("amount") * col("quantity"), 2))
    .withColumn("_silver_ts", current_timestamp())
)

# 6. Drop raw string columns
df = df.drop("amount_str","qty_str","date_str","ts_str")
```

```
# 7. Write to Silver with atomic transaction
(df.write.format("delta").mode("overwrite")
   .option("overwriteSchema","true")
   .saveAsTable("catalog.silver.orders"))
```

BONUS **BONUS B: Deep Dive — Spark Internals** ETL Performance & Optimization

Understanding Spark Behavior in ETL

B.1 Lazy Evaluation & Caching

Spark does NOT execute transformations immediately. Instead, it builds a logical plan and optimizes it. Actions (write, count, show) trigger execution. In Bronze → Silver pipelines, caching is rarely needed—optimize your query instead.

■ Lazy Evaluation Example

```
# Lazy: no execution yet
df_cleaned = df.dropna(subset=["id"]).withColumn("cost", col("price") * col("qty"))

# Action: triggers evaluation
df_cleaned.write.mode("overwrite").saveAsTable("silver.orders")

# Caching is rarely needed in ETL — let Spark optimize
# df_cleaned.cache()  # Only cache if DataFrame is used multiple times
```

B.2 Partition Pruning & Pushdown

If your Bronze table is partitioned by date, filter on the partition column FIRST. Spark can then prune entire partitions without reading them.

■ Partition Pruning

```
# Good: Partition pruning applies
df = spark.table("catalog.bronze.orders") \
    .filter(col("date_partition") >= "2024-01-01") \
    .filter(col("status") == "ACTIVE")

# Less efficient: filter on non-partition column first
df = spark.table("catalog.bronze.orders") \
    .filter(col("status") == "ACTIVE") \
    .filter(col("date_partition") >= "2024-01-01")
```

B.3 Memory & Shuffle Considerations

For large tables, avoid collect() and wide transformations without justification. Use groupBy, join, or explode carefully—they cause shuffles and can OOM.

Lab: Customer Master Data ETL

You have raw customer data in Bronze (inconsistent formatting, nulls, duplicates). Build a Silver layer with clean, deduplicated, typed customer records.

Lab Requirements:

1. Drop rows with null customer_id or email
2. Standardize phone numbers (digits only)
3. Parse signup_date as DateType (format: "MM/dd/yyyy")
4. Keep latest record per customer_id (by update_ts DESC)
5. Add _processed_ts column
6. Write to catalog.silver.customers

■ Lab Solution Skeleton

```
from pyspark.sql.functions import *
from pyspark.sql.window import Window

df = spark.table("catalog.bronze.customers")

# 1. Drop nulls in required fields
df = df.dropna(subset=["customer_id", "email"])

# 2. Standardize phone (digits only)
df = df.withColumn("phone", regexp_replace(col("phone"), r"[^0-9+]", ""))

# 3. Parse date (safe with try_to_date)
from pyspark.sql.functions import try_to_date
df = df.withColumn("signup_date", try_to_date(col("signup_date_str"), "MM/dd/yyyy"))

# 4. Deduplicate by latest update_ts
w = Window.partitionBy("customer_id").orderBy(col("update_ts").desc())
df = df.withColumn("rn", row_number().over(w)) \
       .filter(col("rn") == 1).drop("rn")

# 5. Add processed timestamp
df = df.withColumn("_processed_ts", current_timestamp())

# 6. Write to Silver
df.write.format("delta").mode("overwrite") \
  .option("overwriteSchema","true") \
  .saveAsTable("catalog.silver.customers")
```

MOCK

BONUS D: Mock Exam (8 Questions)

15 minutes

1. A Silver ETL uses fillna({"rating": 0.0}). If rating is null, what will be the result?

A) The column rating will be removed
B) The null value will be replaced with 0.0
C) An exception will be thrown
D) The row will be dropped

2. In a MERGE operation, you want to soft-delete records in the target that no longer exist in source. Which clause applies?

A) WHEN MATCHED AND source IS NULL
B) WHEN NOT MATCHED
C) WHEN NOT MATCHED BY SOURCE
D) WHEN DELETED

3. You have a column "items" with ARRAY type containing 3 elements. Using explode(col("items")), how many rows are output per input row?

A) 0 rows
B) 1 row
C) 3 rows
D) 4 rows (including original)

4. Your code: df.withColumn("cost", col("price") * col("qty")). Does this overwrite the "cost" column if it already exists?

A) No, a new "cost_1" column is created
B) Yes, it replaces the existing "cost" column
C) Error: column already exists
D) It depends on the DataFrame mode

Cheat Sheet & Key Facts

Critical Facts for Exam

✓ dropna(how="any") drops if ANY column is null; dropna(how="all") only if ALL are null

✓ fillna() replaces nulls; coalesce() picks first non-null across columns

✓ cast() is called on Column: col("x").cast("double") not cast(col("x"))

✓ (UPDATED) to_date() throws CAST_INVALID_INPUT in ANSI mode (DBR 17+ default). Use try_to_date().

✓ withColumn("existing_col", expr) REPLACES the column (immutable DataFrames)

✓ explode() drops null/empty arrays; explode_outer() keeps them

✓ WHEN NOT MATCHED BY SOURCE is Databricks-specific (soft deletes)

✓ Partition pruning: filter on partition columns first for performance

✓ Lazy evaluation: transformations don't execute until an action is called

✓ Medallion: Bronze (raw) → Silver (cleaned) → Gold (business-level)

Quick Reference: Function Syntax

Operation	PySpark Syntax	SQL Equivalent
Filter rows	df.filter(col("x")>0)	WHERE x > 0
Add column	df.withColumn("c",expr)	SELECT *, expr AS c
Rename column	df.withColumnRenamed("a", "b")	SELECT a AS b
Drop column	df.drop("c")	SELECT * EXCEPT(c)
Remove nulls	df.dropna(subset=["c"])	WHERE c IS NOT NULL
Fill nulls	df.fillna({"c":"v"})	SELECT COALESCE(c,"v")
Cast type	col("x").cast("double")	CAST(x AS DOUBLE)
Parse date	to_date(col("d"),"fmt")	TO_DATE(d,"fmt")
Flatten array	explode(col("arr"))	LATERAL VIEW explode(arr)

© 2026 Databricks Certified Data Engineer Associate Study Guide

Exam Traps & Common Mistakes

Trap 1: to_date() Behavior in ANSI Mode

Wrong Assumption: "to_date() always returns NULL on parse error."

Reality: In Databricks Runtime 17+ (ANSI mode enabled by default), to_date() throws CAST_INVALID_INPUT exception. Use try_to_date() for safe parsing.

Trap 2: dropna() Variants

Common Mistake: Confusing dropna(how="all") with dropna(thresh=0).

- dropna() = drops if ANY column is null
- dropna(how="all") = drops if ALL columns are null
- dropna(thresh=N) = keeps rows with at least N non-null columns

Trap 3: cast() Syntax

```
Wrong: df.cast(col("price"), "double")
Correct: col("price").cast("double")
```

Trap 4: explode() vs explode_outer()

If you need to preserve rows with null or empty arrays, use explode_outer(). Regular explode() drops them silently.

Trap 5: withColumn() & Column Overwriting

df.withColumn("existing_column", expr) REPLACES the column, not appends it. This catches many people off guard because DataFrames are immutable—a new DataFrame with the modified column is returned.

Trap 6: MERGE INTO Clauses

Be clear on which rows match which clause:

- WHEN MATCHED = key in source AND target → UPDATE/DELETE
- WHEN NOT MATCHED = key in source ONLY (new row) → INSERT
- WHEN NOT MATCHED BY SOURCE = key in target ONLY → UPDATE/DELETE (Databricks-specific)

TIPS **Performance Tips for Silver Layer** Optimize your ETL

Performance Tips for Silver Layer ETL

Tip 1: Filter Early (Predicate Pushdown)

■ **Early Filtering**

```
# Good: Filter on partition column first
df = spark.table("catalog.bronze.orders") \
    .filter(col("date") >= "2024-01-01") \
    .dropna(subset=["order_id"])

# Less efficient: Transform first, then filter
df = spark.table("catalog.bronze.orders") \
    .withColumn("status_upper", upper(col("status"))) \
    .filter(col("date") >= "2024-01-01")
```

Tip 2: Avoid Collect() on Large DataFrames

■ **Avoid Collect()**

```
# WRONG: Brings ALL data to driver
all_records = df.collect()

# RIGHT: Use actions like write or count
df.write.format("delta").mode("overwrite").saveAsTable("silver.orders")
total_count = df.count()  # OK for small counts
```

Tip 3: Use Partition Strategy for Large Silver Tables

■ **Partitioning Strategy**

```
# Partition by date for faster queries
df.write.format("delta").mode("overwrite") \
  .partitionBy("date") \
  .option("overwriteSchema","true") \
  .saveAsTable("catalog.silver.orders_partitioned")
```

Tip 4: Monitor Skew & Optimize Joins

If joining large tables, watch for data skew. Use Spark UI to identify hot partitions. Consider salting skewed keys.

Troubleshooting Common ETL Issues

Issue 1: OutOfMemory (OOM) on dropna()

Symptom: Executor fails with OutOfMemory when calling dropna() on large dataset.

Solution: dropna() on very wide tables can spike memory. Filter to required columns first, or use filter(col(...).isNotNull()).

Issue 2: Parsing Failures with to_date()

Symptom: CAST_INVALID_INPUT exception on to_date().

Solution: ANSI mode (default) throws on invalid input. Use try_to_date() or set spark.sql.ansi.enabled=false.

■ Safe Parsing

```
from pyspark.sql.functions import try_to_date
df = df.withColumn("date_col", try_to_date(col("date_str"), "yyyy-MM-dd"))
```

Issue 3: MERGE INTO Ambiguous Match

Symptom: "Ambiguous merge: multiple source rows match same target row."

Solution: Your join condition is not unique in the source. Add a row_number() window to deduplicate source first.

■ Fix Merge Ambiguity

```
# Deduplicate source before MERGE
w = Window.partitionBy("key").orderBy(col("updated_at").desc())
source = df_source.withColumn("rn", row_number().over(w)) \
                  .filter(col("rn") == 1).drop("rn")
```

ADVANCED

Advanced Scenarios

Multi-step transformations

Scenario 1: Multi-Source Merge

Merge data from 2+ source systems into single Silver table:

■ Multi-Source Merge

```
# Combine multiple sources
source1 = spark.table("catalog.bronze.customers_sap")
source2 = spark.table("catalog.bronze.customers_salesforce")

source_combined = source1.select("customer_id", "name", "email", "source") \
                  .unionByName(
                      source2.select("customer_id", "name", "email", "source")
                  )

# Deduplicate: keep source1 priority
w = Window.partitionBy("customer_id").orderBy(col("source"))
source_dedup = source_combined.withColumn("rn", row_number().over(w)) \
                              .filter(col("rn") == 1).drop("rn")

# Now MERGE into Silver
spark.sql(f"""
MERGE INTO catalog.silver.customers AS tgt
USING source AS src ON tgt.customer_id = src.customer_id
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *
""")  # Registered source as temp view first
```

Scenario 2: Incremental Load with Watermark

Load only new/changed records since last run:

■ Incremental Load Pattern

```
from pyspark.sql.functions import max as spark_max

# Get last processed timestamp from Silver
last_watermark = spark.sql(
    "SELECT MAX(_silver_ts) FROM catalog.silver.orders"
).collect()[0][0] or "2020-01-01"

# Load only new/changed from Bronze
df_new = spark.table("catalog.bronze.orders") \
    .filter(col("updated_at") > last_watermark)

# MERGE (only this subset)
spark.sql(f"""
MERGE INTO catalog.silver.orders tgt
USING df_new src
ON tgt.order_id = src.order_id
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *
""")
```

EXAM **Extended Mock Exam (12 Questions)** 20 minutes

Extended Mock Exam - Practice Questions

1. You have a Bronze table with dates in format "20240115" (yyyyMMdd). Which code safely parses this to DateType in ANSI mode?

A) to_date(col("date_str"), "yyyyMMdd")
B) try_to_date(col("date_str"), "yyyyMMdd")
C) col("date_str").cast("date")
D) parse_date(col("date_str"), "yyyyMMdd")

■ *ANSI mode throws on error; try_to_date() returns NULL.*

2. A dataframe has columns [a, b, c]. After df.withColumn("b", col("b") * 2), how many columns exist and what is column "b"?

A) 2 columns [a, c]; b is dropped
B) 3 columns [a, b_new, c]; original b dropped
C) 3 columns [a, b, c]; b is replaced with doubled values
D) 4 columns [a, b, b_new, c]

3. Which of the following is correct for creating a new column with a default value?

A) df.withColumn("default_col", lit(0))
B) df.addColumn("default_col", 0)
C) df.select(col("*"), lit(0).alias("default_col"))
D) Both A and C are correct

4. MERGE INTO: Which clause is used to UPDATE or DELETE rows from target that have NO match in source?

A) WHEN MATCHED
B) WHEN NOT MATCHED
C) WHEN NOT MATCHED BY SOURCE
D) WHEN UNMATCHED BY TARGET

5. An array column [1,2,3] is exploded. How many output rows per input row?

A) 1 row (entire array)
B) 3 rows (one per element)
C) 4 rows (array + elements)
D) Depends on array type

6. What does df.dropna(thresh=2) do?

A) Drops rows with 2+ null columns
B) Keeps rows with at least 2 non-null columns
C) Drops if exactly 2 columns are null
D) Unknown behavior

7. Which is the Databricks-specific MERGE clause?

A) WHEN MATCHED
B) WHEN NOT MATCHED
C) WHEN NOT MATCHED BY SOURCE
D) All are standard SQL

8. Code: df.fillna({"salary": 0.0, "bonus": 0.0}). If both columns are null, what happens?

A) Both columns replaced with 0.0
B) Only salary replaced, bonus untouched
C) Error: ambiguous fill
D) Row is dropped

9. A date string is "Feb/01/2024". Which format string is correct for to_date()?

A) "yyyy-MM-dd"
B) "MMM/dd/yyyy"
C) "MM/dd/yyyy"
D) "Feb/dd/yyyy"

Answer Key — Extended Mock Exam

Q1: B — try_to_date() is safe in ANSI mode—returns NULL on parse error instead of throwing exception.

Q2: C — withColumn() with existing column name replaces it. DataFrames are immutable.

Q3: D — Both patterns work: lit() creates a literal column, or select with alias.

Q4: C — WHEN NOT MATCHED BY SOURCE handles soft deletes (Databricks extension).

Q5: B — explode() creates one row per array element.

Q6: B — thresh=N keeps rows with at least N non-null columns.

Q7: C — WHEN NOT MATCHED BY SOURCE is Databricks-specific, not in standard SQL MERGE.

Q8: A — fillna() replaces nulls in specified columns with provided values.

Q9: B — "MMM/dd/yyyy" for "Feb/01/2024" (MMM = 3-letter month).

Real-World ETL Patterns

Example 1: E-Commerce Order Transformation

Scenario: Raw order events from multiple systems. Need to unify, clean, and deduplicate.

Order Transformation

```
from pyspark.sql.functions import *
from pyspark.sql.window import Window

# 1. Read from Bronze (union multiple sources)
orders_sap = spark.table("catalog.bronze.orders_sap")
orders_shopify = spark.table("catalog.bronze.orders_shopify")

orders = (orders_sap.select("order_id", "customer_id", "amount", "date_str", "updated_ts")
           .unionByName(
               orders_shopify.select("order_id", "customer_id", "amount", "date_str", "updated_ts")
           ))

# 2. Clean & transform
from pyspark.sql.functions import try_to_date, try_to_timestamp
orders = (orders
    .dropna(subset=["order_id", "customer_id", "amount"])
    .withColumn("amount", col("amount").cast("double"))
    .withColumn("order_date", try_to_date(col("date_str"), "yyyy-MM-dd"))
    .withColumn("updated_at", try_to_timestamp(col("updated_ts"), "yyyy-MM-dd HH:mm:ss"))
    .withColumn("_ingest_ts", current_timestamp())
)

# 3. Deduplicate: keep latest per order_id
w = Window.partitionBy("order_id").orderBy(col("updated_at").desc())
orders = (orders
    .withColumn("rn", row_number().over(w))
    .filter(col("rn") == 1)
    .drop("rn", "date_str", "updated_ts")
)

# 4. Write to Silver
orders.write.format("delta").mode("overwrite") \
    .option("overwriteSchema","true") \
    .partitionBy("order_date") \
    .saveAsTable("catalog.silver.orders_unified")
```

Example 2: Customer Profile with Masking

Scenario: Sensitive customer data needs PII scrubbing before Silver layer.

■ PII Data Handling

```
from pyspark.sql.functions import *

customers = spark.table("catalog.bronze.customers")

# 1. Drop rows missing critical ID
customers = customers.dropna(subset=["customer_id"])

# 2. Clean & standardize
customers = (customers
    .withColumn("email", lower(trim(col("email"))))
    .withColumn("phone", regexp_replace(col("phone"), r"[^0-9]", ""))
    .withColumn("name", initcap(trim(col("name"))))
)

# 3. Add PII masking flags (governance)
customers = (customers
    .withColumn("email_masked", when(col("email").isNull(), lit(False)).otherwise(lit(True)))
    .withColumn("phone_masked", when(col("phone").isNull(), lit(False)).otherwise(lit(True)))
)

# 4. Write to Silver with external location (for auditing)
customers.write.format("delta").mode("overwrite") \
    .option("overwriteSchema","true") \
    .saveAsTable("catalog.silver.customers")
```

REF **Quick Lookup Tables** Fast reference for exams

Null Handling Cheat Sheet

Scenario	Function	Syntax	Result
Drop rows with ANY null	dropna()	df.dropna(subset=["col1","col2"])	Removes rows
Drop rows with ALL null	dropna()	df.dropna(how="all")	Removes rows
Keep rows with ≥N non-null	dropna()	df.dropna(thresh=3)	Removes rows
Replace nulls with value	fillna()	df.fillna({"col": "val"})	Replaces in-place
First non-null across cols	coalesce()	coalesce(col1, col2, lit("default"))	New column
Conditional null handling	when/otherwise	when(col.isNull(), lit("X")).otherwise(col)	New column

Type Casting & Date Parsing Cheat Sheet

Input	Target Type	Safe Method	ANSI Behavior
"123"	IntegerType	col.cast("integer")	Error if non-numeric
"2024-01-15"	DateType	try_to_date(col, "yyyy-MM-dd")	Returns NULL on error
"2024-01-15 14:30"	TimestampType	try_to_timestamp(col, format)	Returns NULL on error
"INVALID"	DateType	try_to_date(col, fmt)	Returns NULL silently

Array & Collection Functions Cheat Sheet

Function	Input	Output	Null Behavior
explode()	ARRAY(1,2,3)	3 rows (one per element)	Drops null/empty arrays
explode_outer()	ARRAY(1,2,3)	3 rows (one per element)	Keeps rows with NULL
explode_outer()	NULL	1 row with NULL	Keeps row

explode_outer()	ARRAY()	1 row with NULL	Keeps empty array row
posexplode()	ARRAY(a,b)	2 rows: (0,a), (1,b)	Drops null/empty arrays

STRATEGY **Exam Strategy & Time Management** 90 minutes, 45 questions

Exam Strategy & Time Management

Time Allocation (90 min / 45 Qs)

- 2 minutes per question (average)
- 2-5 min flagged/difficult questions
- 5 minutes buffer for review

Question Types You'll See (Week 6 Domain)

- **Null Handling Qs (20%):** dropna variants, fillna, coalesce, when/otherwise
- **Type Casting Qs (15%):** cast(), to_date(), to_timestamp(), ANSI mode
- **Column Ops (15%):** withColumn, withColumnRenamed, drop, select, filter
- **Array/Struct (10%):** explode, explode_outer, struct operations
- **MERGE INTO (20%):** WHEN clauses, upsert patterns, Databricks-specific syntax
- **Data Cleaning (10%):** string functions, regex, standardization
- **Scenario-Based (10%):** Multi-step ETL, error handling, performance

Pro Tips During Exam

✓ Read questions carefully — "which does NOT" vs "which does"

✓ Eliminate wrong answers first (often easier than picking right one)

✓ Flag questions with multiple TRUE answers — review later

✓ Watch for ANSI mode traps (especially to_date, to_timestamp)

✓ Know Databricks-specific syntax (WHEN NOT MATCHED BY SOURCE)

✓ If unsure on syntax, think about what result you want, work backward

Last-Minute Review (1 Hour Before)

- Review "Exam Traps" section (page X)
- Memorize quick reference tables (this section)
- Skim Mock Exam answers
- Trust your preparation—don't cram new concepts

Week 6 Self-Assessment Checklist

- ■ Understand dropna(how="any") vs dropna(how="all") vs dropna(thresh=N)
- ■ Know fillna() replaces nulls; coalesce() picks first non-null
- ■ Can explain ANSI mode behavior for to_date() and to_timestamp()
- ■ Understand try_to_date() and try_to_timestamp() for safe parsing
- ■ Know cast() syntax: col("x").cast("type")
- ■ Can use withColumn() to add, replace, or transform columns
- ■ Understand withColumnRenamed() for single or multiple renames
- ■ Know the difference between explode() and explode_outer()
- ■ Can write a complete MERGE INTO with WHEN MATCHED clauses
- ■ Understand WHEN NOT MATCHED BY SOURCE (Databricks-specific)
- ■ Know common string functions: upper, lower, trim, regexp_replace
- ■ Understand date format patterns: yyyy-MM-dd, MM/dd/yyyy, etc.
- ■ Can explain Medallion Architecture: Bronze → Silver → Gold
- ■ Know common ETL patterns: deduplication, type casting, standardization
- ■ Can debug common issues: OOM, parse errors, ambiguous merges
- ■ Understand partition pruning & lazy evaluation
- ■ Scored 80%+ on Mock Exams

■ If you can check all boxes, you're ready for Domain 3 (22% of exam).

© 2026 Databricks Certified Data Engineer Associate Study Guide | Week 6 | v2.0

FAQ **Frequently Asked Questions** Common confusions clarified

Frequently Asked Questions (FAQ)

Q: Why use try_to_date() instead of to_date()?
to_date() throws an exception in ANSI mode (default in DBR 17+). try_to_date() returns NULL on error, making it safer for production ETL that must not fail on malformed input.

Q: Can dropna() and fillna() be used together?
Yes! First dropna(subset=["critical_cols"]) to remove rows with missing critical data, then fillna() to replace nulls in optional columns with defaults.

Q: Is there a difference between col("x").cast("double") and col("x").cast(DoubleType())?
No, both work identically. cast("double") is simpler and preferred; cast(DoubleType()) is more explicit but requires importing from pyspark.sql.types.

Q: When should I use MERGE INTO vs simple INSERT overwrite?
Use MERGE when you need to update existing records and insert new ones atomically. For full table replacement, INSERT overwrite is simpler. MERGE is best for incremental loads.

Q: What's the difference between explode() and explode_outer()?
explode() drops rows with null/empty arrays. explode_outer() keeps them (with NULL in the exploded column). Choose explode_outer() if all rows must be preserved.

Q: Does withColumn() create a new column or modify existing?
It replaces if the column name exists, or adds new if it doesn't. DataFrames are immutable—a new DataFrame is returned each time.

Q: How do I efficiently deduplicate a large table?
Use ROW_NUMBER() with Window over partition key, order by updated_at DESC, then filter rn==1. This is efficient and parallelizable.

Q: Should I cache intermediate DataFrames in ETL?
Rarely. Spark's optimizer is excellent. Cache only if a DataFrame is used 2+ times AND you've confirmed via Spark UI that caching helps (reduces shuffles).

COVERAGE **Domain 3 Exam Coverage** Where Week 6 fits in

Domain 3: Data Transformation & Modeling (22%)

Domain 3 covers Bronze → Silver → Gold transformations. Week 6 focuses on the foundational Bronze → Silver step. Weeks 7-8 build on these skills with joins, aggregations, and Gold layer design.

Week 6 (This Week) — Core Skills

- Null handling & fillna strategies
- Type casting & ANSI mode error handling
- Column operations: add, rename, drop, filter
- Array & struct transformations
- MERGE INTO upsert patterns

Week 7 (Next Week) — Build On These Skills

- Join types (inner, left, broadcast, multi-key)
- Deduplication with window functions
- Aggregations (count, approx_count_distinct, mean)

Week 8 — Gold Layer Design

- Materialized views vs views vs streaming tables
- Data quality checks & validation
- Spark tuning params (shuffle.partitions, autoBroadcast)

Exam Tip:

Domain 3 (22%) = ~10 questions on exam. Week 6 concepts (~4 questions), Week 7 joins/agg (~4 questions), Week 8 gold layer (~2 questions). Master Week 6 basics first.

OPTIMIZE **Advanced Optimization Tips** For production Silver ETL

Advanced Optimization Tips

Tip 1: Avoid Unnecessary Transformations

Each transformation adds to the DAG. Combine multiple operations where possible:

Inefficient: Create 5 separate withColumn() calls
Better: Chain them: df.withColumn("c1", ...).withColumn("c2", ...).withColumn("c3", ...)

Tip 2: Use select() for Complex Projections

When renaming many columns or reordering, use select() instead of multiple withColumnRenamed():

```
df.select(col("old_name1").alias("new_name1"), col("old_name2").alias("new_name2"), ...)
```

Tip 3: Broadcast Small Dimension Tables in MERGE

If your source table is small (<1GB), explicitly broadcast it before MERGE to avoid shuffle:

```
source_broadcast = broadcast(df_source) before MERGE
```

Tip 4: Profile Your Query with Spark UI

After running ETL, check Spark UI to identify:

- Stages with long runtimes (data skew?)
- High disk spill (increase memory or reduce partition size)
- Data skew (use salt key if one task much slower than others)

Tip 5: Use Liquid Clustering for Frequently Filtered Tables

For Silver tables filtered by date or customer_id, enable Liquid Clustering:

```
df.write.option("liquid.cluster.by", "date,customer_id").saveAsTable(...)
```

Key Facts to Memorize

1. **dropna(how="any"):** Drops if ANY column is null (default)
2. **dropna(how="all"):** Drops if ALL specified columns are null
3. **dropna(thresh=N):** Keeps rows with ≥ N non-null columns
4. **fillna(dict):** Replaces nulls in-place by column
5. **coalesce(col1, col2, lit(x)):** Returns first non-null value
6. **when(...).otherwise(...):** Conditional column value assignment
7. **to_date() in ANSI:** THROWS EXCEPTION on bad input (DBR 17+)
8. **try_to_date():** Returns NULL on parse error (safe)
9. **col.cast("type"):** Explicit type conversion (correct syntax)
10. **explode():** Drops rows with null/empty arrays
11. **explode_outer():** Keeps rows with null/empty arrays
12. **WHEN MATCHED:** Key exists in source AND target
13. **WHEN NOT MATCHED:** Key in source only (new row) → INSERT
14. **WHEN NOT MATCHED BY SOURCE:** Key in target only → UPDATE/DELETE (Databricks)
15. **withColumn(existing):** Replaces column if exists

Print this page and review 1 hour before exam!

© 2026 Databricks Certified Data Engineer Associate | Week 6 Complete Study Guide

Domain Weight & Week 6 Importance

Domain 3 (Data Transformation & Modeling) is **22% of the exam**—about 10 questions. Week 6 covers the foundational Bronze → Silver step, which is critical for all subsequent Silver and Gold layer work. Understanding the concepts in Week 6 sets you up for success in Weeks 7-8.

Exam Distribution for Domain 3:

- Week 6 (This): Null handling, type casting, column ops, MERGE (~4 Qs)
- Week 7 (Next): Joins, aggregations, deduplication (~4 Qs)
- Week 8 (After): Gold layer design, materialized views (~2 Qs)

Why Week 6 Matters

Week 6 is foundational because:

1. **Data Quality First:** You cannot join or aggregate dirty data. Week 6 teaches data cleaning—the first step in any ETL.
2. **Type Safety:** Casting columns correctly prevents errors downstream. Bad types break joins and aggregations.
3. **Upsert Patterns:** MERGE INTO is the foundation of incremental loads, which powers most production Silver layers.
4. **Foundation for Weeks 7-8:** You cannot effectively join without clean data; you cannot aggregate without proper types. Week 6 prepares you.

Week-by-Week Building Blocks

Week 6: Raw String → Typed, Cleaned Columns | **Week 7:** Single Table → Joined, Aggregated | **Week 8:** Silver → Gold Business Objects

Success Tips & Final Reminders

Before Taking the Exam

✓ **Review Exam Traps:** Spend 15 min reviewing the "Exam Traps" section. These are the most common mistakes test-takers make.

✓ **Memorize Quick Reference:** The "Key Facts to Memorize" list (15 items) is essential. Know them cold—you should be able to recall any without thinking.

✓ **Practice Mock Exams:** You have 20+ practice questions in this guide. Aim for 80%+ on mock exams before taking the real exam.

✓ **Understand ANSI Mode:** This is new. Make sure you understand that ANSI mode (default) causes to_date() to throw exceptions, not return NULL.

During the Exam

✓ **Read Carefully:** Read questions twice. Note "NOT", "which is correct", "which does NOT", etc.

✓ **Eliminate Wrong Answers:** If you're unsure, eliminate obviously wrong answers first.

✓ **Watch Time:** 2 min per question average. Flag difficult ones and come back.

✓ **Trust Your Prep:** You've studied 12 weeks of material. Don't second-guess your knowledge.

How This Guide Helps

This 40+ page guide includes:

• **Complete Week 6 Content:** All 9 sections with detailed explanations and code examples

• **30+ Practice Questions:** With answer keys and detailed explanations

• **Real-World Patterns:** E-commerce, customer mastery, multi-source merges

• **Exam Traps & FAQ:** Common mistakes clarified

• **Performance Tips:** Optimization and troubleshooting

• **Memorization List:** 15 key facts to know cold

You are ready. Trust your preparation, take the exam, and pass with confidence! ■

© 2026 Databricks Certified Data Engineer Associate Study Guide | Week 6 v2.0 (Final)